

Rapport de Projet : Compilateur C-

Soni DIEDHIOU & Bilal KEFIF

1^{er} décembre 2025

Table des matières

1	Introduction	2
2	Architecture du Projet	2
3	Détails Techniques et Choix d'Implémentation	2
3.1	L'Analyse Lexicale (Lexer)	2
3.2	L'Analyse Syntaxique (Parser)	2
3.3	Gestion des Déclarations Multiples	2
3.4	Analyses Sémantiques	3
3.5	Modélisation de l'AST et Types Options	3
3.6	Optimisations Syntaxiques et Raffinement de l'AST	3
3.7	Gestion des Pointeurs et Arithmétique des Types	3
4	Fonctionnement et Utilisation	4
4.1	Compilation du projet	4
4.2	Exécution	4
5	Conclusion	4

1 Introduction

Ce projet consiste en la réalisation d'un compilateur pour un sous-ensemble du langage C, nommé "C-". L'objectif est de passer par toutes les étapes classiques de la compilation : analyse lexicale, analyse syntaxique, construction de l'Arbre Syntaxique Abstrait (AST) et analyses sémantiques (portée et typage).

Le compilateur est écrit en OCaml, utilisant `ocamllex` pour le lexer et `Menhir` pour le parser.

2 Architecture du Projet

Le projet est découpé en plusieurs modules, chacun ayant une responsabilité unique dans la chaîne de compilation :

- `ast.ml` : Définit la structure de données de l'Arbre Syntaxique Abstrait. C'est le cœur du projet, définissant les types `expr` (expressions), `instr` (instructions) et `ctype` (types C).
- `lexer.mll` : Le fichier source pour `ocamllex`. Il transforme le flux de caractères en une suite de *tokens* (mots du langage).
- `parser.mly` : Le fichier source pour `Menhir`. Il définit la grammaire du langage et les règles de construction de l'AST à partir des tokens.
- `check.ml` : Effectue l'analyse de portée (Scope). Il vérifie que chaque variable utilisée a bien été déclarée au préalable.
- `types.ml` : Effectue l'analyse de typage. Il vérifie la cohérence des opérations (ex : ne pas additionner un entier et une chaîne).
- `main.ml` : Le point d'entrée du programme. Il orchestre l'appel aux différents modules :
Lexer → Parser → Check → Types.

3 Détails Techniques et Choix d'Implémentation

3.1 L'Analyse Lexicale (Lexer)

Le lexer utilise des expressions régulières pour identifier les éléments du langage. Un point technique intéressant est la gestion des nombres flottants. Nous avons dû définir plusieurs règles (`float_1`, `float_2`, `float_3`) pour couvrir tous les cas valides en C :

- Classique : `3.14`
- Sans partie entière : `.5`
- Scientifique sans point : `1e10`

Cela permet une robustesse que n'aurait pas une expression régulière unique trop complexe.

3.2 L'Analyse Syntaxique (Parser)

Le parser utilise une grammaire non contextuelle. Nous avons utilisé les directives de priorité (`%left`, `%right`, `%nonassoc`) pour résoudre les ambiguïtés classiques, comme la priorité de la multiplication sur l'addition, ou le problème du "dangling else" (résolu avec `%nonassoc THEN/ELSE`).

Une particularité technique est l'utilisation de règles récursives pour gérer les listes (ex : `list_top_decl`). Le parser construit ces listes élément par élément (tête : : queue) lors de la remontée récursive.

3.3 Gestion des Déclarations Multiples

Un défi technique a été la gestion des déclarations de type C comme `int *x, y;`. Ici, `x` est un pointeur sur `int`, mais `y` est un `int` simple. **Solution** : Nous avons séparé la définition du type de base (`type_spec`) de la liste des déclarateurs.

- `type_spec` renvoie le type de base (`int`).

- `declarator_list` renvoie une liste de paires (`nom`, `niveau_pointeur`).
- La règle `decl` combine les deux pour associer le bon type complet à chaque variable.

3.4 Analyses Sémantiques

Nous avons séparé l'analyse en deux passes distinctes pour plus de clarté :

1. **Check (Scope)** : Utilise une liste de tables de hachage (`Map`) pour gérer les environnements imbriqués. À l'entrée d'un bloc `{ ... }`, on ajoute une nouvelle table en tête de liste. À la sortie, on la retire, ce qui gère automatiquement la portée des variables locales.
2. **Types** : Parcourt l'AST annoté ou l'environnement pour vérifier la compatibilité des types (ex : compatibilité `int` vs `float`, déréférencement de pointeurs).

3.5 Modélisation de l'AST et Types Options

Pour représenter fidèlement la syntaxe du C où certains éléments sont facultatifs, nous avons tiré parti du type `option` d'OCaml. Par exemple, l'instruction `if` est définie comme :

```
| IIIf of expr * instr * instr option
```

Cela permet de distinguer explicitement un `if` sans `else` (`None`) d'un `if` avec `else` (`Some instr`). De même pour les boucles `for` (initialisation, condition, incrémentation optionnelles) et le `return` (avec ou sans valeur). Cette modélisation force le compilateur à gérer tous les cas de figure, évitant des bugs d'oubli.

3.6 Optimisations Syntaxiques et Raffinement de l'AST

Le parser ne se contente pas de construire une représentation brute du code ; il effectue plusieurs simplifications à la volée pour produire un AST plus compact et optimisé :

- **Pliage des constantes négatives** : L'opérateur moins unaire appliqué à une constante (ex : `-5`) est détecté par la grammaire et converti directement en une valeur `CInt(-5)`. Sans cette règle spécifique, le parser produirait une opération de soustraction `0 - 5`, ce qui alourdirait l'arbre inutilement.
- **Fusion des chaînes de caractères** : Conformément au standard C, les littéraux chaînes adjacents (ex : `"Hello " "World"`) sont concaténés dès l'analyse syntaxique en un unique nœud `CStr`. Cela délègue cette complexité au parser plutôt qu'au générateur de code.

Ces micro-optimisations montrent l'intérêt d'utiliser un parser intelligent capable de prétraiter certaines constructions syntaxiques.

3.7 Gestion des Pointeurs et Arithmétique des Types

Un aspect critique de la compilation du C est la gestion des niveaux d'indirection (pointeurs). Dans notre AST, chaque type possède un champ `pointer` (entier) indiquant son niveau d'indirection (0 pour une variable, 1 pour `*`, 2 pour `**`).

Le module de typage gère dynamiquement ce niveau lors des opérations :

- L'opérateur d'adresse `&x` incrémente le niveau de pointeur du type de `x` ($T \rightarrow T^*$).
- L'opérateur de déréférencement `*p` décrémente ce niveau ($T^* \rightarrow T$). Une vérification est effectuée pour empêcher le déréférencement d'une variable qui n'est pas un pointeur (niveau 0).

Cette modélisation par un simple compteur entier simplifie grandement la logique de validation des types complexes comme `int ***p`.

4 Fonctionnement et Utilisation

4.1 Compilation du projet

Le projet utilise un `Makefile`. Pour compiler le compilateur `cmoins`, il suffit d'exécuter :

```
make
```

Cela génère l'exécutable `cmoins`. Pour nettoyer les fichiers générés :

```
make clean
```

4.2 Exécution

Pour compiler/analyser un fichier source C- :

```
./cmoins chemin/vers/fichier.c
```

Le programme affichera :

1. L'AST reconstruit (via `Printer`).
2. Les erreurs éventuelles (Lexicale, Syntaxique, Portée ou Typage).
3. Un message de succès si tout est correct.

5 Conclusion

Ce projet a permis de mettre en pratique les concepts théoriques de la compilation. La structure modulaire (Lexer/Parser/Check/Types) permet une maintenance aisée et une évolution progressive du langage. Les choix techniques, notamment sur la représentation de l'AST et la gestion fine de la grammaire, assurent une analyse robuste du code source.